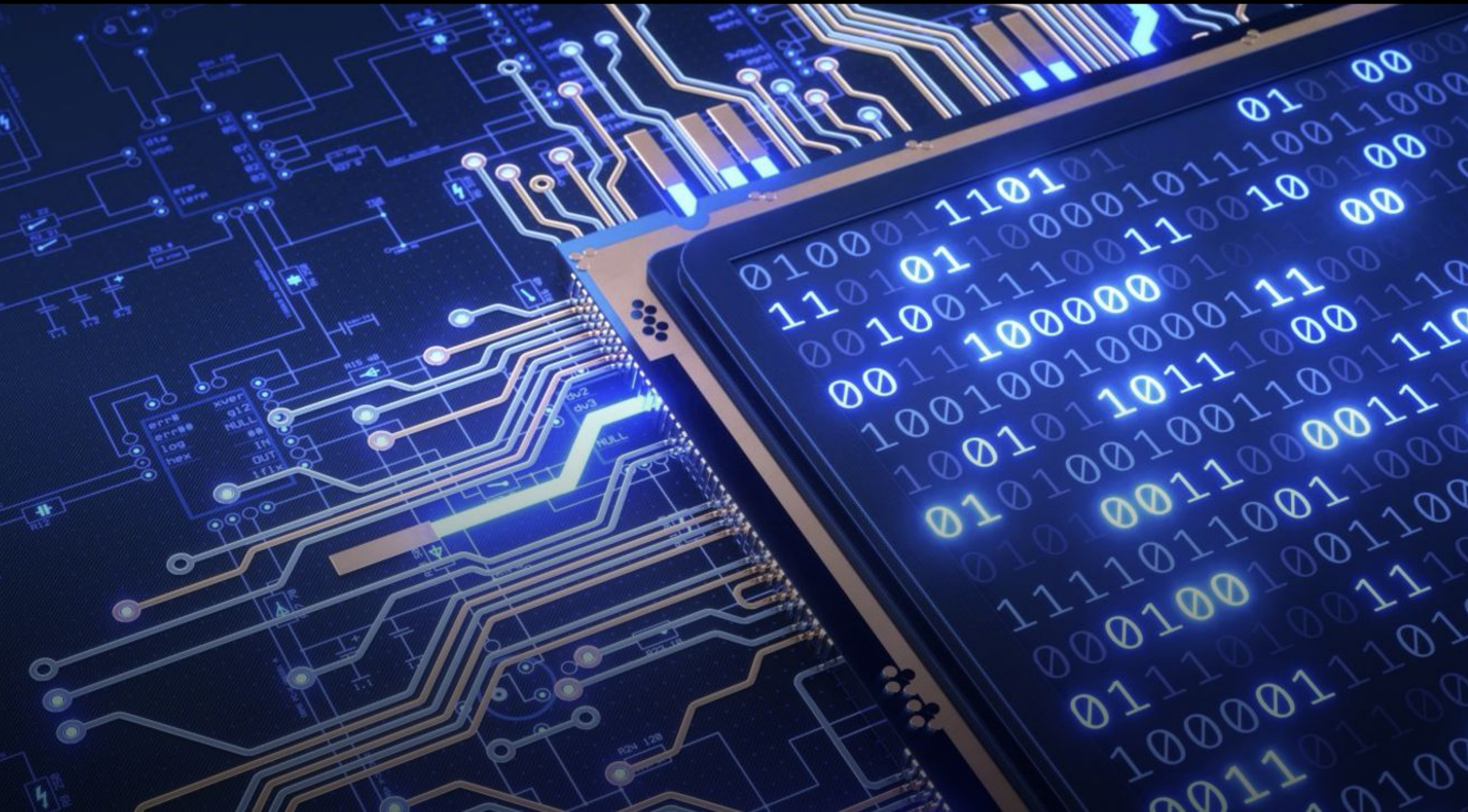


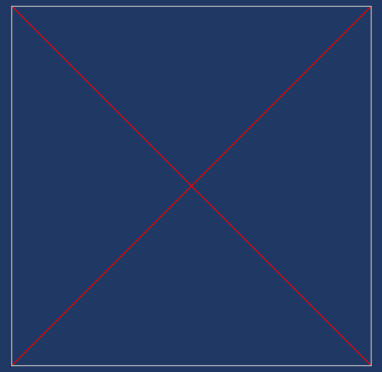
Prototipazione di un Cancello Automatico: Un Approccio Embedded

A cura di:
Cicatelli Antonio – M63001746
D'Avanzo Gianni – M63001714
Gentile Carmine – M63001816
Gentile Enrico – M63001788

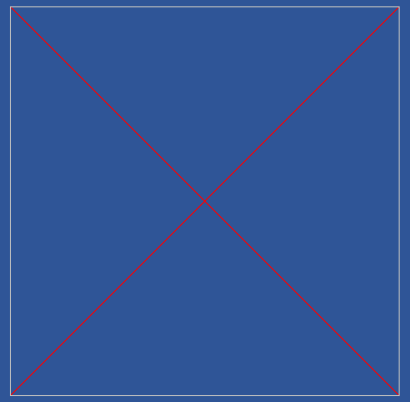
OBIETTIVI

Prototipazione di un Cancello Automatico: Un Approccio Embedded

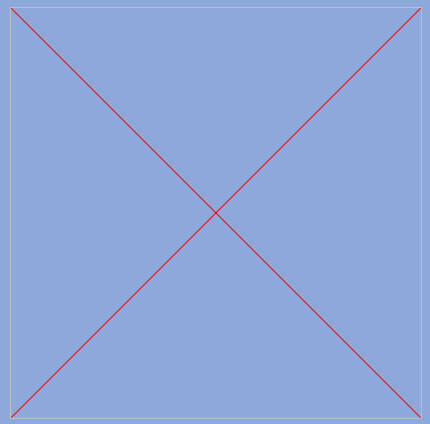
A cura di:
Cicatelli Antonio – M63001746
D'Avanzo Gianni – M63001714
Gentile Carmine – M63001816
Gentile Enrico – M63001788



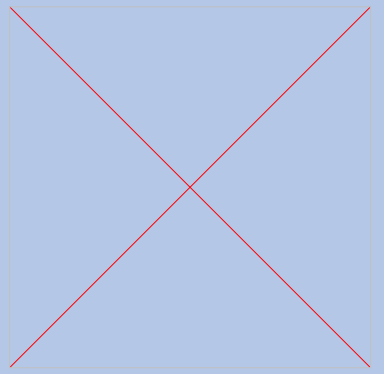
OBIETTIVI



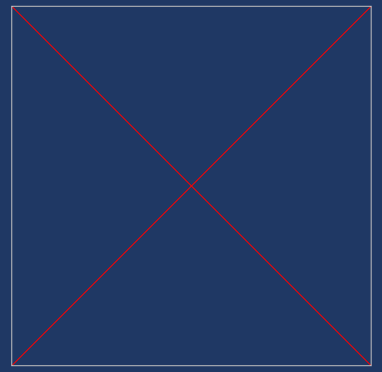
PROGETTAZIONE



IMPLEMENTAZIONE



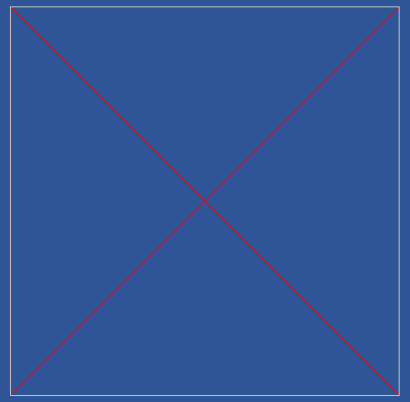
TEST & VALIDAZIONE



OBIETTIVI

FINALITA’ DEL PROGETTO

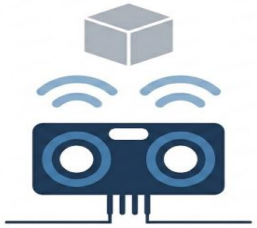
- Controllo Wireless di un cancello automatico
- Rilevamento di ostacoli Real-Time
- Movimento coordinato delle due ante
- Sicurezza e integrità della comunicazione



PROGETTAZIONE

ARCHITETTURA BASE

RILEVAMENTO

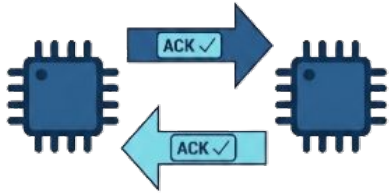


ARRESTO DEL MOVIMENTO

TRASMISSIONE

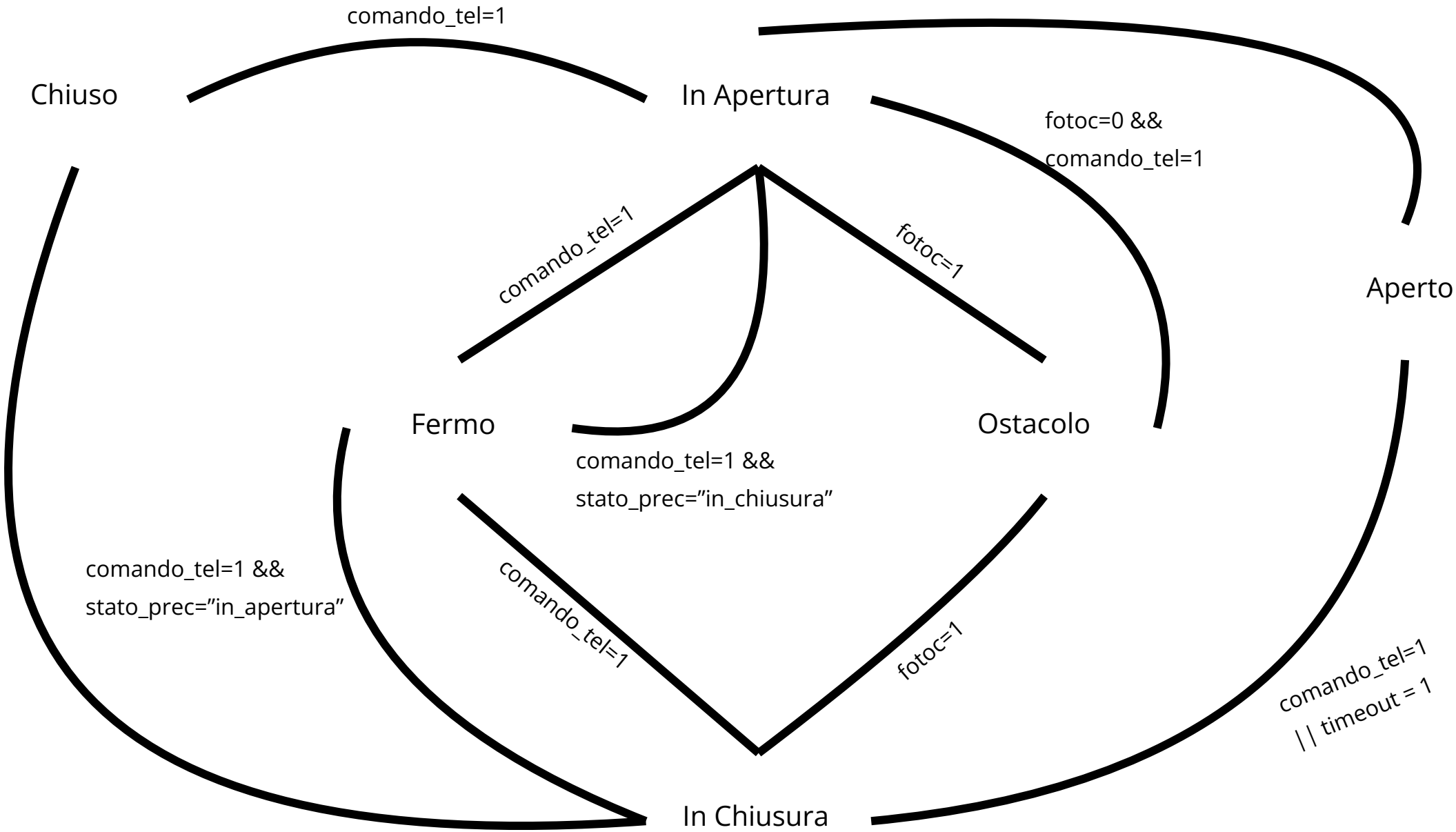
SICUREZZA ED INTEGRITÀ

SINCRONIZZAZIONE



MOVIMENTO SICURO E COORDINATO

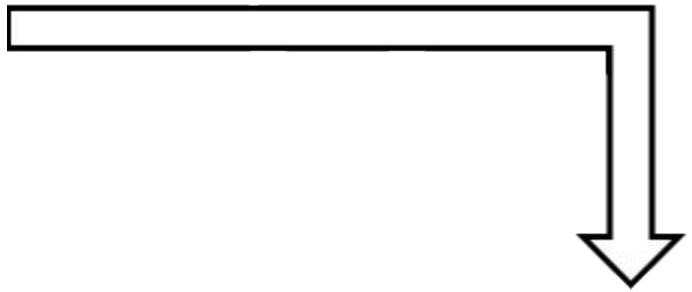
MACCHINA A STATI FINITI


Chiuso
comando_tel=1
In Apertura
fotoc=0 &&
comando_tel=1
Aperto
comando_tel=1
fotoc=1
Fermo
Ostacolo
comando_tel=1 &&
stato_prec="in_chiusura"
comando_tel=1 &&
stato_prec="in_apertura"
comando_tel=1
fotoc=1
comando_tel=1
|| timeout = 1
In Chiusura

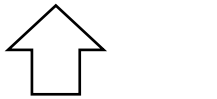
MECCANISMO DI SICUREZZA

REPLY ATTACKS

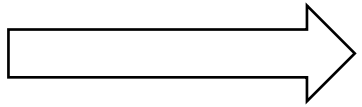
ID: 1, Contatore: 10



'L'ho già visto questo pacchetto. Lo Ignoro'



ID: 1, Contatore: 10

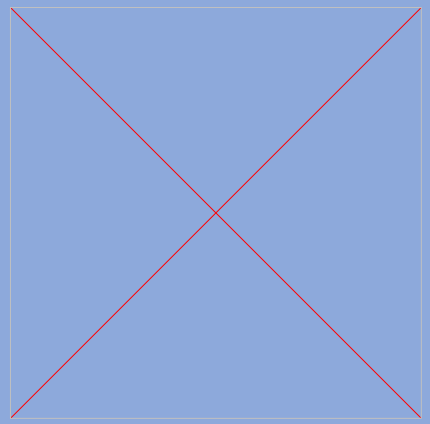


ID NON AUTORIZZATO

ID: 1000, Contatore: 1

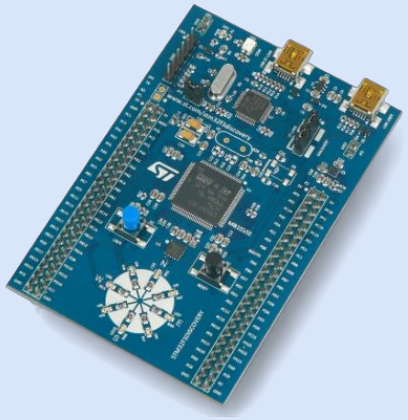
'Questo Telecomando non è memorizzato. Lo Ignoro'

ID: 1, Contatore: 10

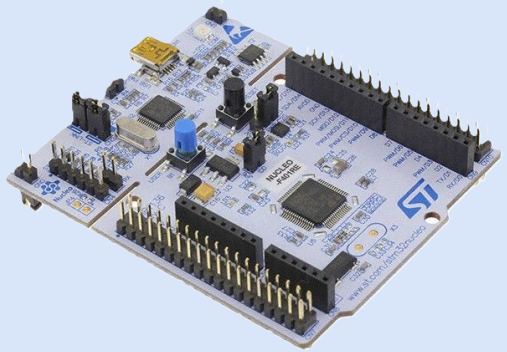


IMPLEMENTAZIONE

X-Nucleo IDS01A4

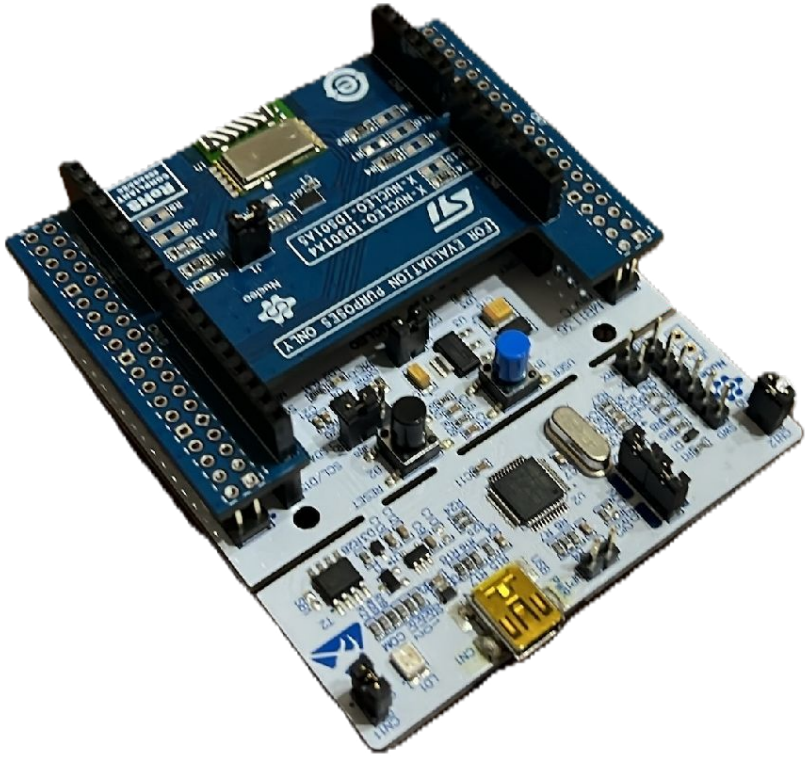


STM32F3DISCOVERY



STM32F2401RE

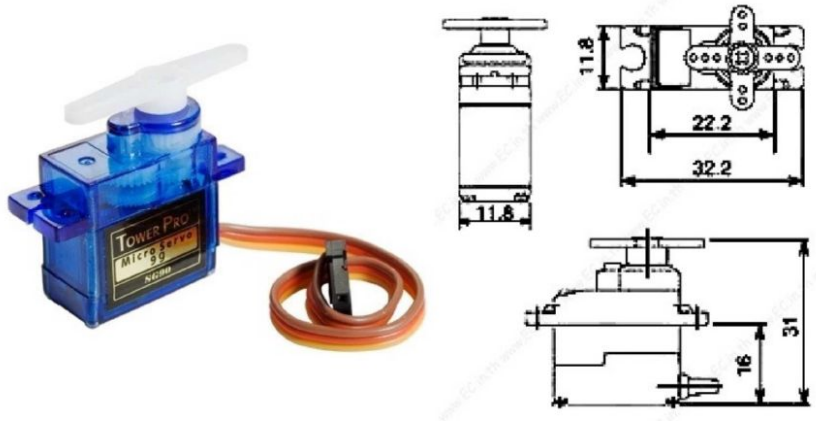
X-Nucleo IDS01A4



- Permette la comunicazione wireless tra i nodi del sistema.
- La comunicazione con il microcontrollore avviene tramite SPI.
- L'utilizzo della banda Sub-GHz a 868 MHz garantisce un'ottima gestione degli ostacoli.

Servomotore Sg90

SG90 9 g Micro Servo

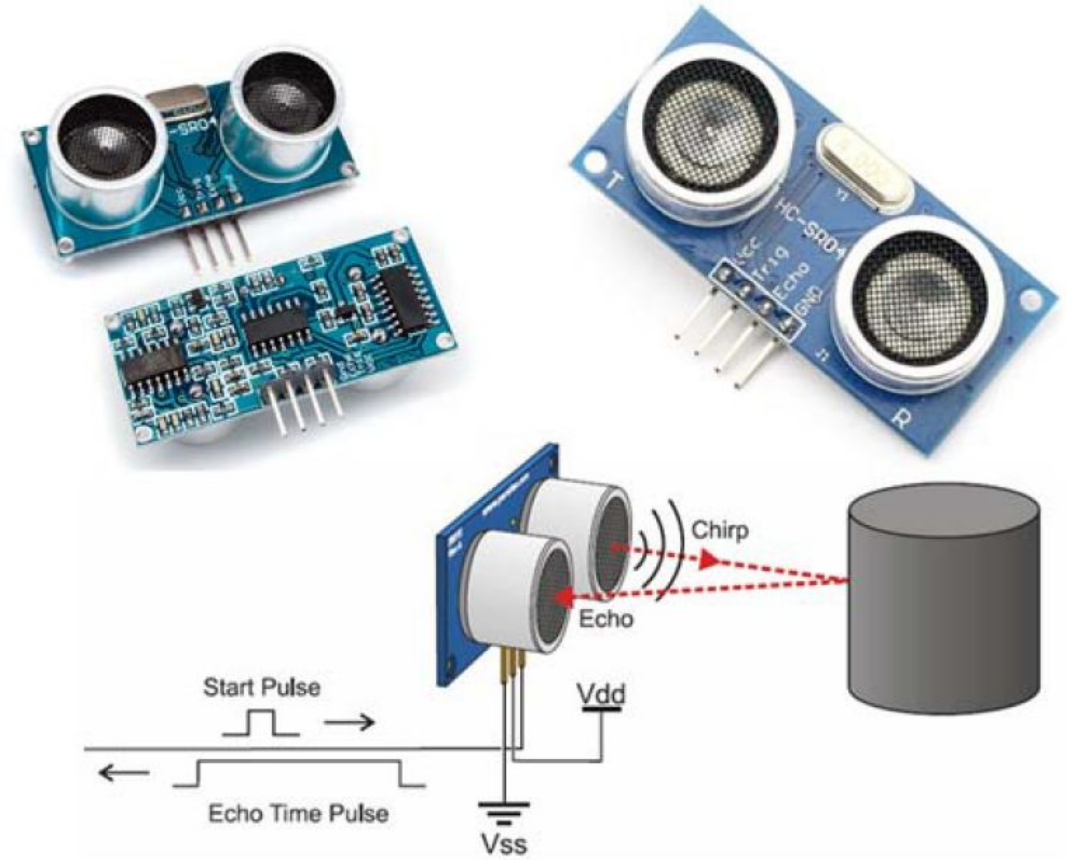


Specifications

- Weight: 9 g
- Dimension: 22.2 x 11.8 x 31 mm approx.
- Stall torque: 1.8 kgf·cm
- Operating speed: 0.1 s/60 degree
- Operating voltage: 4.8 V (~5V)
- Dead band width: 10 µs
- Temperature range: 0 ºC – 55 ºC

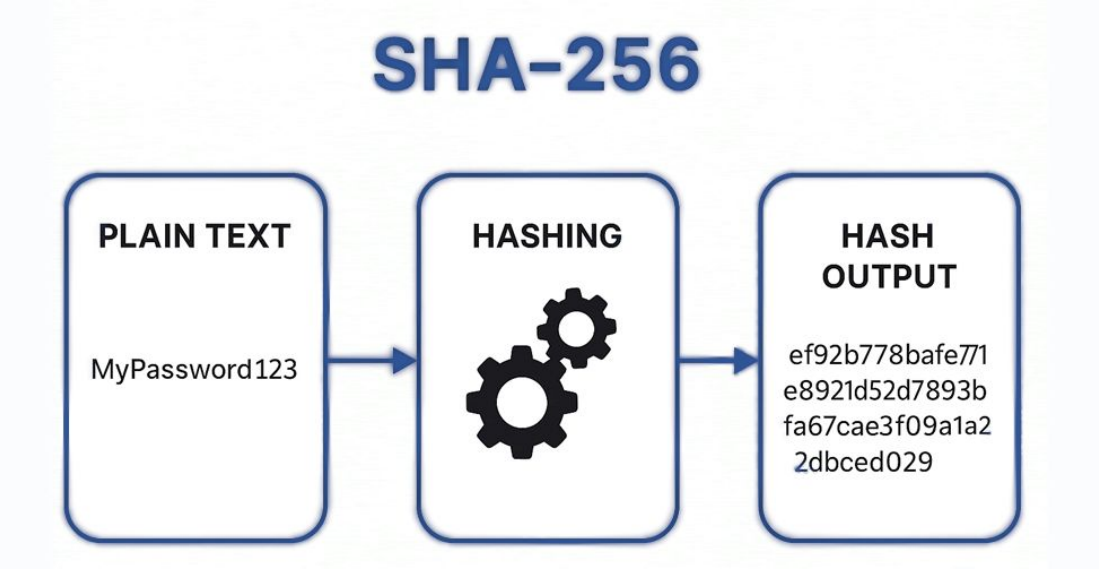
- Permette il movimento (apertura e chiusura) delle ante.
- Pilotato tramite un segnale PWM a 50Hz generato direttamente dalla scheda STM32.
- Il movimento si basa sulla lunghezza dell’impulso fornito.

Sensore Ultrasuoni HC-SR04



- Monitora in tempo reale l'area di transito durante il movimento.
- Il sensore viene attivato tramite un breve segnale Trigger generato dalla scheda.
- Calcola la distanza dell’ostacolo tramite il tempo di ritorno del segnale Echo.

Cyber-Sicurezza: Sha256

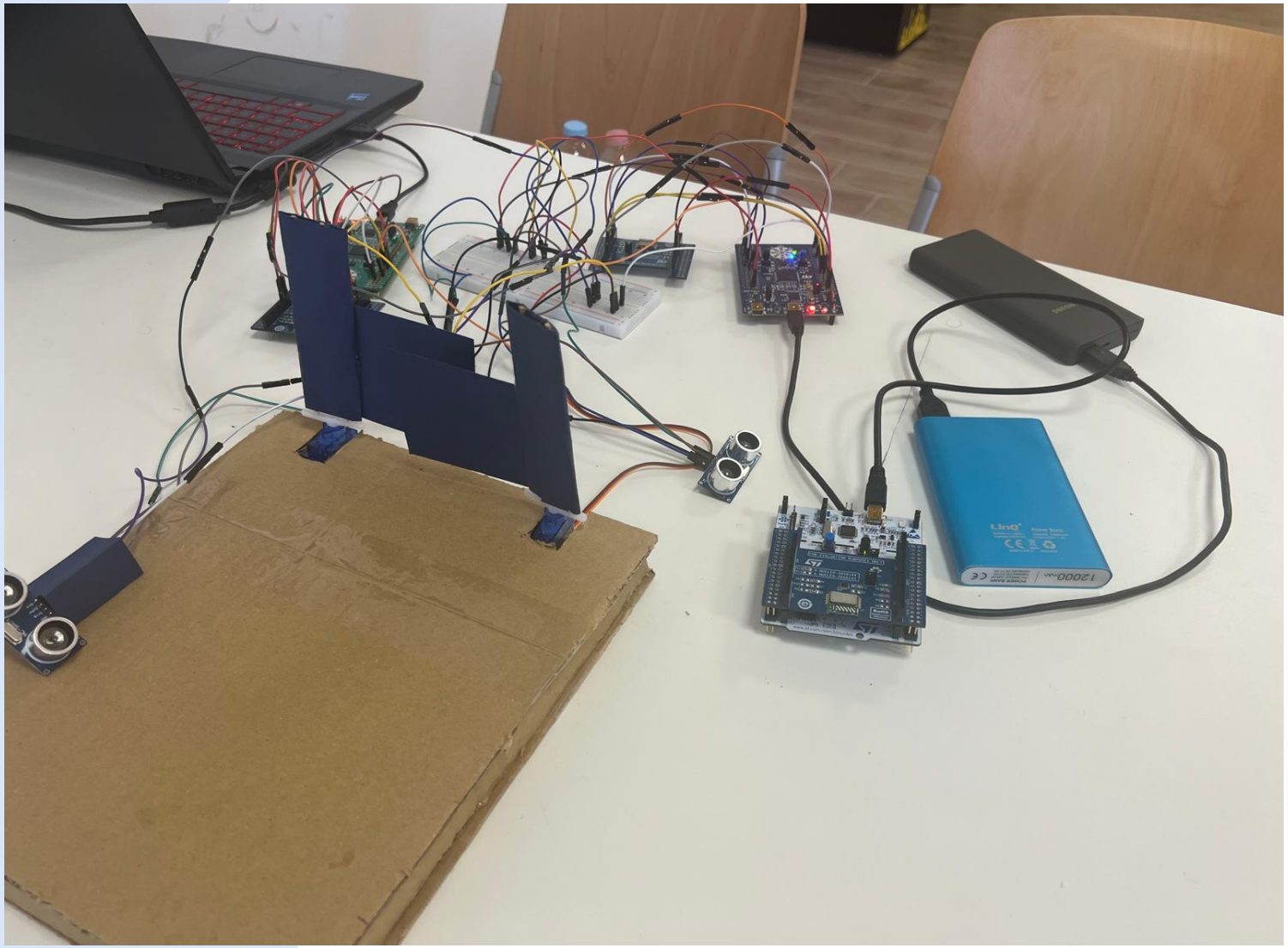


- Genera una stringa univoca sulla base di ID, Contatore e chiave segreta.
- Inviata insieme ai dati in chiaro (Contatore, ID) garantiamo confidenzialità e integrità.
- Versione custom e small per renderla accessibile alla potenza di calcolo ridotta del microcontrollore.

COLLEGAMENTI HARDWARE

ID	Componente A (Sorgente)	Pin Componente A	Funzione	Componente B (Destinazione)	Pin Componente B
1	STM32F3-Discovery	PA6	MISO	X-Nucleo IDS01A4	PA6
2	STM32F3-Discovery	PA7	MOSI	X-Nucleo IDS01A4	PA7
3	STM32F3-Discovery	PA10	SDN	X-Nucleo IDS01A4	PA10
4	STM32F3-Discovery	PB3	CLOCK	X-Nucleo IDS01A4	PB3
5	STM32F3-Discovery	PB6	CHIP SELECT	X-Nucleo IDS01A4	PB6
6	STM32F3-Discovery	PC7	INTERRUPT	X-Nucleo IDS01A4	PC7
7	STM32F3-Discovery	PB11-PB12	ECHO	Ultrasuoni HC-SR04	ECHO
8	STM32F3-Discovery	PD4-PD5	TRIGGER	Ultrasuoni HC-SR04	TRIG
9	STM32F3-Discovery	PA15	PWM	Servo Sg90	PWM

ARCHITETTURA FINALE



IMPLEMENTAZIONE SOFTWARE

Macchina a Stati Finiti

```
void Muovi_Cancello(){

    switch (stato_attuale) {
            case CHIUSO:
                pwm_bersaglio = 1000;
                if (comando_tel) {
                    stato_attuale = IN_APERTURA;
                    comando_tel = 0;
                }
                break;

            case IN_APERTURA:
                pwm_bersaglio = 1800;
                if (fotoc) {
                    stato_attuale = OSTACOLO;
                }
                else if (pwm_attuale == 1800) {
                    stato_attuale = APERTO;

                    __HAL_TIM_SET_COUNTER(&htim4, 0);
                    __HAL_TIM_CLEAR_FLAG(&htim4, TIM_FLAG_UPDATE);
                    HAL_TIM_Base_Start_IT(&htim4);
                }

                else if (comando_tel) {
                    stato_precedente = IN_APERTURA;
                    stato_attuale = FERMO;
                    comando_tel = 0;
                }
                break;

            case APERTO:
                pwm_bersaglio = 1800;
                if (comando_tel) {
                    stato_attuale = IN_CHIUSURA;
                    comando_tel = 0;

                    __HAL_TIM_SET_COUNTER(&htim4, 0);
                    HAL_TIM_Base_Stop_IT(&htim4);
                }
                break;

            case IN_CHIUSURA:
                pwm_bersaglio = 1000;
                if (fotoc) {
                    stato_attuale = OSTACOLO;
                }
                else if (pwm_attuale == 1000) {
                    stato_attuale = CHIUSO;
                }

                else if (comando_tel) {
                    stato_precedente = IN_CHIUSURA;
                    stato_attuale = FERMO;
                    comando_tel = 0;
                }
                break;

            case FERMO:
                pwm_bersaglio = pwm_attuale;
                if (comando_tel) {
                    stato_attuale = (stato_precedente == IN_CHIUSURA) ? IN_APERTURA : IN_CHIUSURA;
                    comando_tel = 0;
                }
                break;

            case OSTACOLO:
                pwm_bersaglio = pwm_attuale;
                if (!fotoc && comando_tel) {
                    HAL_Delay(100);
                    stato_attuale = IN_APERTURA;
                    comando_tel = 0;
                }
                break;
}
```

Fotocellula

```
void Fotocellula(){

    if (HAL_GetTick() - ultimo_trigger >= 50) {
        ultimo_trigger = HAL_GetTick();

        if (turno_sensore == 0) {
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, GPIO_PIN_SET);
            for(volatile int i = 0; i < 100; i++);
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, GPIO_PIN_RESET);
            turno_sensore = 1;
        } else {
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_5, GPIO_PIN_SET);
            for(volatile int i = 0; i < 100; i++);
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_5, GPIO_PIN_RESET);
            turno_sensore = 0;
        }
    }

    if (eco_ricevuto_sx == 1) {
        eco_ricevuto_sx = 0;
        if (tempo_ritorno_eco <= 2000) {
            contatore_ostacolo_sx++;
        } else {
            contatore_ostacolo_sx = 0;
        }

        if (contatore_ostacolo_sx >= 3) fotoc_sx = 1;
        else fotoc_sx = 0;
    }

    if (eco_ricevuto_dx == 1) {
        eco_ricevuto_dx = 0;
        if (tempo_ritorno_eco <= 2000) {
            contatore_ostacolo_dx++;
        } else {
            contatore_ostacolo_dx = 0;
        }

        if (contatore_ostacolo_dx >= 3) fotoc_dx = 1;
        else fotoc_dx = 0;
    }
```

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{

    if (GPIO_Pin == GPIO_PIN_11) {

        if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_11) == GPIO_PIN_SET) {
            __HAL_TIM_SET_COUNTER(&htim3, 0);
        }
        else {
            tempo_ritorno_eco = __HAL_TIM_GET_COUNTER(&htim3);
            eco_ricevuto_sx = 1;
        }
    }

    if (GPIO_Pin == GPIO_PIN_12) {
        if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_12) == GPIO_PIN_SET) {
            __HAL_TIM_SET_COUNTER(&htim3, 0);
        } else {

            tempo_ritorno_eco = __HAL_TIM_GET_COUNTER(&htim3);

            if (tempo_ritorno_eco > 100) {
                eco_ricevuto_dx = 1;
            }
        }
    }
}
```

Telecomando

```
uint32_t Init_Counter_From_Flash(void) {
    uint32_t last_saved_value = 0;
    uint32_t *flash_ptr = (uint32_t *)FLASH_USER_START_ADDR;

    while (flash_ptr < (uint32_t *)FLASH_USER_END_ADDR) {
        if (*flash_ptr == 0xFFFFFFFF) {
            break;
        }
        last_saved_value = *flash_ptr;
        flash_ptr++;
    }

    if (last_saved_value > 0) {
        return last_saved_value + 15;
    }

    return 0;
}
```

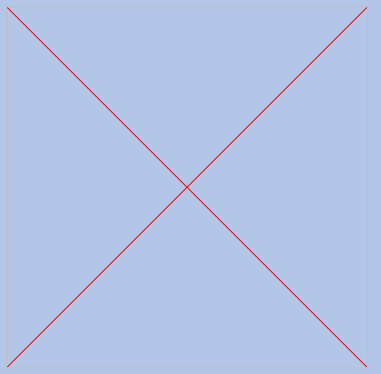
```
void P2P_Init(void)
{

    contatore = Init_Counter_From_Flash();
#if defined(X_NUCLEO_IDS01A4) || defined(X_NUCLEO_IDS01A5)
  DestinationAddr=MULTICAST_ADDRESS;
  pRadioDriver->GpioIrq(&xGpioIRQ);
  pRadioDriver->RadioInit(&xRadioInit);
  pRadioDriver->SetRadioPower(POWER_INDEX, POWER_DBM);
#endif
```

```
void Save_Counter_To_Flash(uint32_t current_counter) {
    uint32_t *flash_ptr = (uint32_t *)FLASH_USER_START_ADDR;
    uint32_t write_addr = FLASH_USER_START_ADDR;

    while (flash_ptr < (uint32_t *)FLASH_USER_END_ADDR) {
        if (*flash_ptr == 0xFFFFFFFF) {
            break;
        }
        flash_ptr++;
        write_addr += 4;
    }

    HAL_FLASH_Unlock();

    if (write_addr >= FLASH_USER_END_ADDR) {
        FLASH_EraseInitTypeDef EraseInitStruct;
        uint32_t SectorError;

        EraseInitStruct.TypeErase = FLASH_TYPEERASE_SECTORS;
        EraseInitStruct.VoltageRange = FLASH_VOLTAGE_RANGE_3;
        EraseInitStruct.Sector = FLASH_USER_SECTOR;
        EraseInitStruct.NbSectors = 1;

        HAL_FLASHEx_Erase(&EraseInitStruct, &SectorError);

        write_addr = FLASH_USER_START_ADDR;
    }

    HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, write_addr, current_counter);

    HAL_FLASH_Lock();
}
```

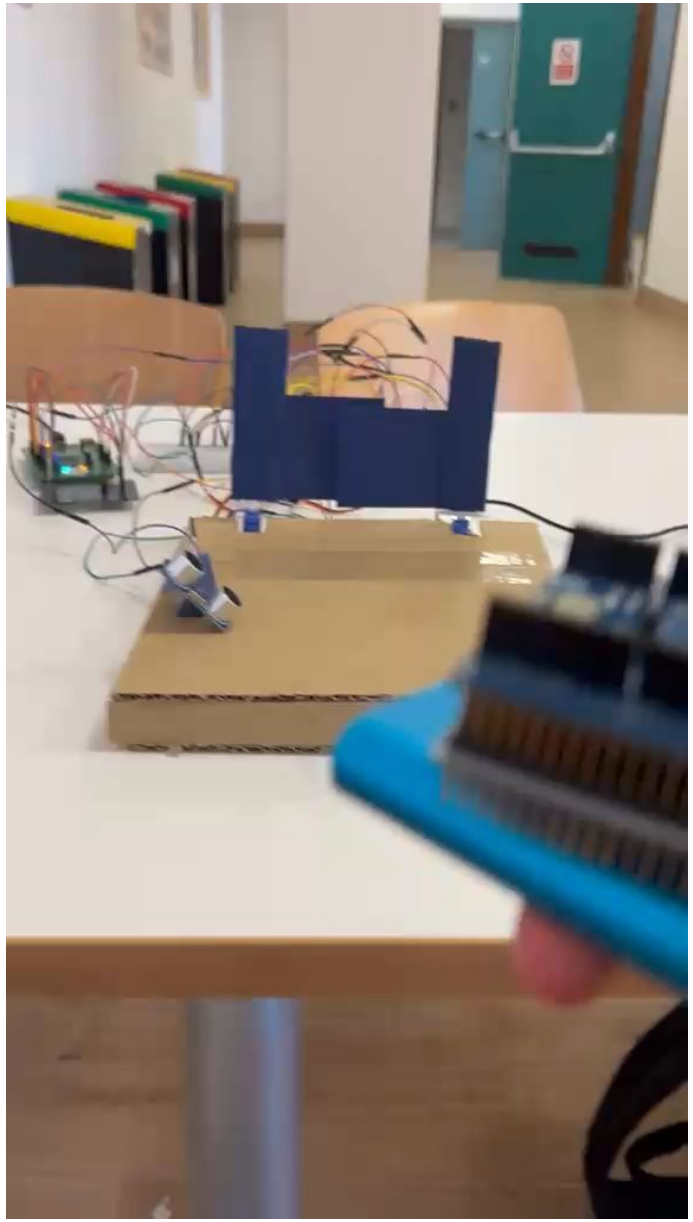
Meccanismo di Sicurezza

```
case SM_STATE_SEND_DATA:
  {
    contatore ++;

    if ((contatore % 15) == 0 || (primo_salvataggio == 1)) {
              Save_Counter_To_Flash(contatore);
              primo_salvataggio = 0;
    }

    memcpy(&pTxBuff[0], &id_telecomando, 4);
    memcpy(&pTxBuff[4], &contatore, 4);

    hmac_sha256(
        CHIAVE_SEGRETA,
        32,
        pTxBuff,
        8,
        &pTxBuff[8]
    );

    xTxFrame.Cmd = LED_TOGGLE;
    xTxFrame.CmdLen = 0x01;
    xTxFrame.Cmdtag = txCounter++;
    xTxFrame.CmdType = APPLI_CMD;
    xTxFrame.DataBuff = pTxBuff;
    xTxFrame.DataLen = cTxlen;

    for (int i = 0; i < 3; i++){
        AppliSendBuff(&xTxFrame, xTxFrame.DataLen);
    }

    SM_State = SM_STATE_WAIT_FOR_TX_DONE;
  }
  break;
```

```
  case SM_STATE_DATA_RECEIVED:
    {
#if defined(X_NUCLEO_S2868A1) || defined(X_NUCLEO_S2915A1)
      pRadioDriver = &radio_cb;
#endif

      pRadioDriver->GetRxPacket(pRxBuff,&cRxlen);

      xRxFrame.Cmd = pRxBuff[0];
      xRxFrame.CmdLen = pRxBuff[1];
      xRxFrame.Cmdtag = pRxBuff[2];
      xRxFrame.CmdType = pRxBuff[3];
      xRxFrame.DataLen = pRxBuff[4];

      if (messaggio_ricevuto == 1 && pRxBuff[0] == ACK_OK){

          messaggio_ricevuto = 0;
          flag_cancello = 1;
          SM_State = SM_STATE_ACK_RECEIVED;

          return;
      }

      static uint8_t memoria_payload_rx[96];
      xRxFrame.DataBuff = memoria_payload_rx;

      for (xIndex = 5; xIndex < cRxlen; xIndex++)
      {
        xRxFrame.DataBuff[xIndex] = pRxBuff[xIndex];
      }

      uint32_t valori_ricevuti[2];
      uint32_t id_ricevuto;
      uint32_t contatore_ricevuto;
      uint8_t firma_calcolata[32];

      int indice_telecomando = -1;

      memcpy(&valori_ricevuti, &pRxBuff[5 + 0], 8);

      for (int i = 0; i < num_telecomandi_registrati; i++) {
              if (Lista_Telecomandi[i].id_telecomando == valori_ricevuti[0]) {
                  indice_telecomando = i;

                  break;
              }
      }

      if (indice_telecomando == -1){
          SM_State = SM_STATE_START_RX;
          messaggio_scartato = 1;
          break;
      }

      if (Lista_Telecomandi[indice_telecomando].contatore >= valori_ricevuti[1]){
          SM_State = SM_STATE_START_RX;
          messaggio_scartato = 1;
          break;
      }

      hmac_sha256(CHIAVE_SEGRETA, 32,valori_ricevuti, 8, firma_calcolata);
      if (memcmp(firma_calcolata, &pRxBuff[8 + 5], 32) == 0) {

          Lista_Telecomandi[indice_telecomando].contatore = valori_ricevuti[1];
          messaggio_ricevuto = 1;
          HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_10);


      } else {
          messaggio_scartato = 1;
          SM_State = SM_STATE_START_RX;
          break;

      }
```



TEST E VALUTAZIONE



PROBLEMATICHE RISCONTRATE

PROBLEMATICHE RISCONTRATE

PROBLEMATICHE RISCONTRATE

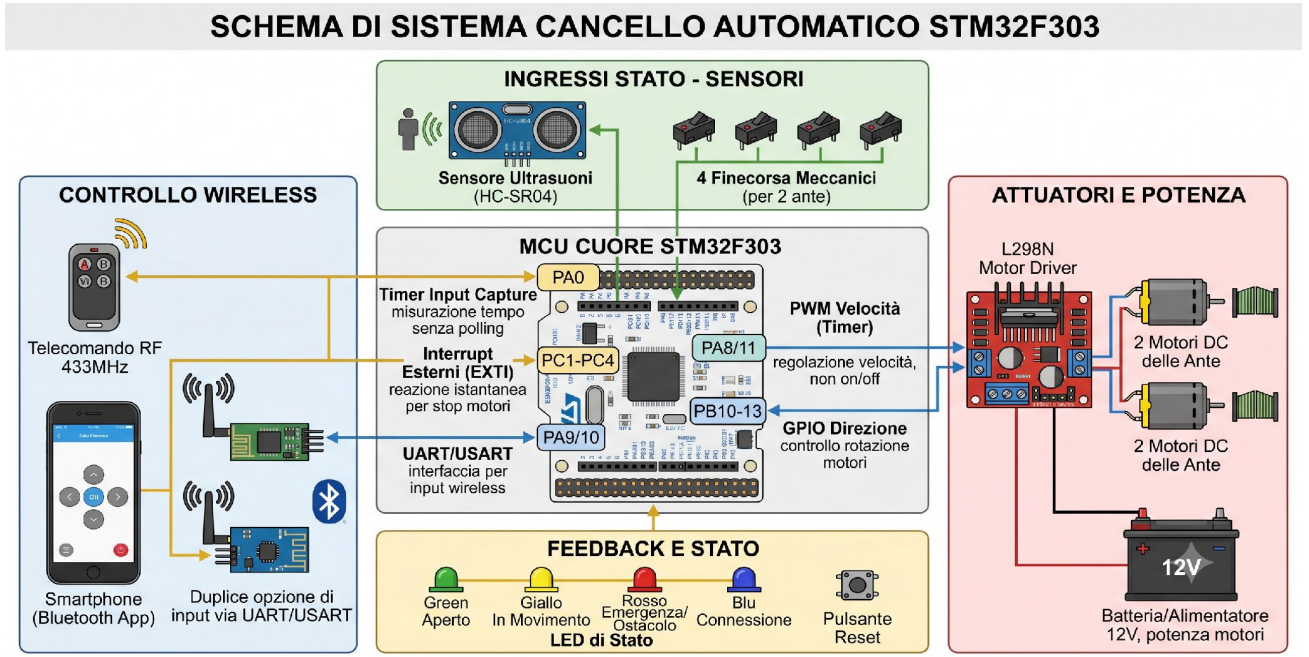
PROBLEMATICHE RISCONTRATE

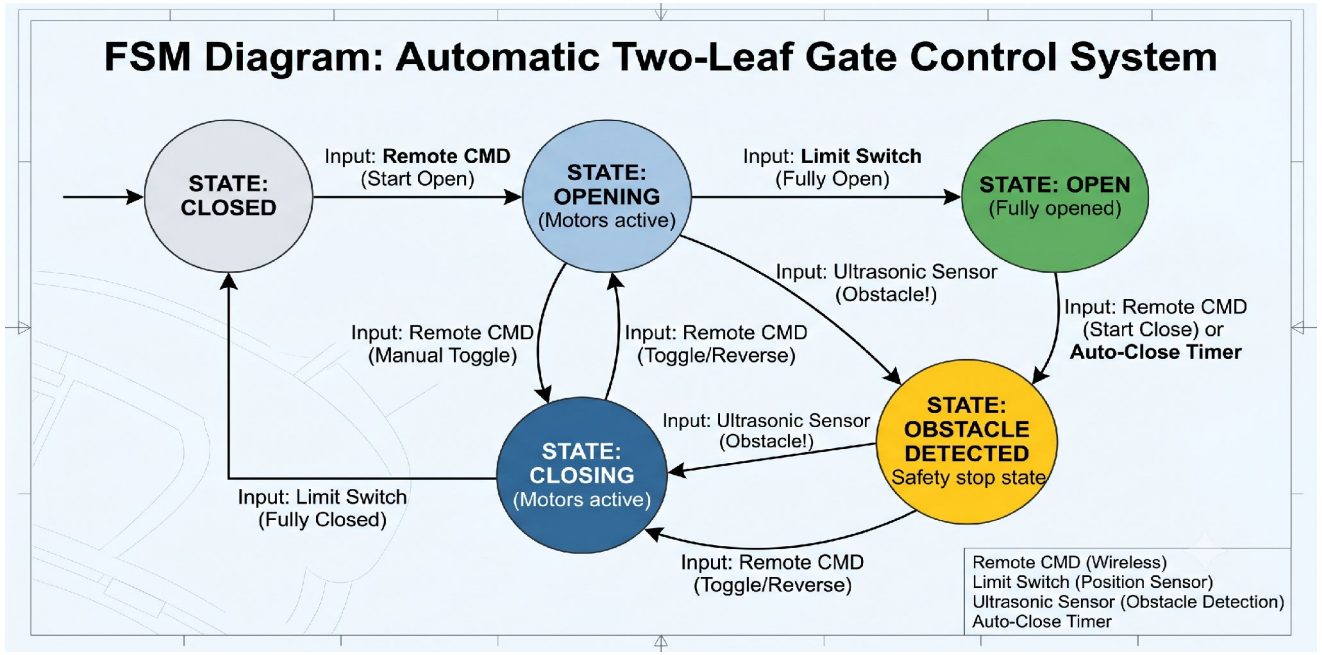
- PORTING DEL FIRMWARE
- POSIZIONAMENTO ULTRASUONI

PROBLEMATICHE RISCONTRATE

- PORTING DEL FIRMWARE
- POSIZIONAMENTO ULTRASUONI
- INGEGNERIZZAZIONE

IA GENERATIVA: SOLUZIONE PROPOSTA





CONCLUSIONI E SVILUPPI FUTURI

- Mitigazione Attacchi DoS e Rate Limiting.
- Sicurezza basata su assorbimento elettrico.
- Porting universale.


ENRICO GENTILE
CARMINE GENTILE
GIANNI D'AVANZO
ANTONIO CICATELLI

GRAZIE PER L’ATTENZIONE